

Validation and Transformation in Backend Systems

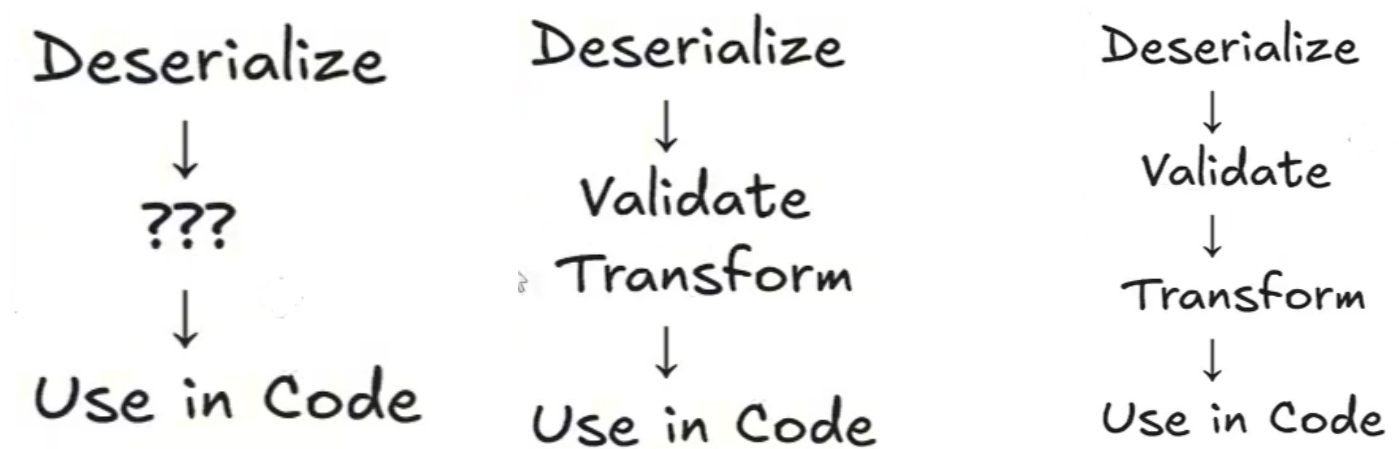
Validation and Transformation

Deserialization Is Not Enough

So, we talked about serialization, deserialization of data, how data crosses a boundary and then comes to be in object memory again. But, here's the part that people miss. We assume it looks something like this, right? But, that raises a question we need to account for. Is this deserialized data even usable? Just because we have deserialized data, that doesn't automatically mean that it can be used. Because deserialization at a fundamental level just gives us a structure of that data, but it doesn't necessarily ensure correctness. A dangerous assumption to make is assuming that deserialized data is automatically safe. And this is an assumption that leads to a lot of back-end bugs and a lot of security issues. Incoming data should always be untrusted.

Like I said, any data that crosses a boundary, that automatically means that the trust of that data goes back down to zero. It doesn't matter if that comes from a browser, another service, or even your own database. Deserialization gives us the structure, but it gives no insight to the meaning or intent of the data. The only question it answers is what is the shape of this data? It doesn't answer whether it makes sense or not.

Validation and Transformation



So, that introduces a need for more steps after this deserialization. We need something in between when we deserialize and when we use the code. Actually, we need two things. We need to **validate** and **transform** this data before we can use it in our code. And actually, it should look something like this. We validate and then we transform it.

And of course, that means *raw input does not automatically mean that this is a usable input*. Back-end systems don't run on raw input when we receive some sort of data. They run on assumptions, and validation and transformation is how we enforce those assumptions. So, I want you guys to recognize that **deserialization alone is simply not enough when we're acquiring data from an outside source**. The serialization is what gets the data into our system, but the validation and transformation actually helps us to understand how we're going to use that data. And if we even can use it.

So now that we know why these steps exist, we need to separate them because obviously they're not the same thing and a common theme in this series, obviously we have two things we're talking about. Going to go ahead and break down both of them.

Validation vs. Transformation

So these two ideas can very commonly be mixed together, but when they do, systems get messy. Raw reason rovers, these hoes love Chief Sosa. I'm listening to Chief Keef right now.

Validation

So validation is simply a gate. It answers one question. **Is this data acceptable in my system?** And it's a yes or no, reject or continue.

Examples:
Required fields?
Correct types?
Within limits?

So here are some examples of things we need to check in validation. We need to see if it has the required fields necessary for the input? Does it have the correct types and is it within the limitations of whatever we're going to throw it into?

If validation fails, then naturally the request fails as well. The request is going to end and nothing else is going to run.

Transformation

Transformation on the other hand answers the question of **what form do I want this data in?** Here we can do things like **normalize** or **convert** or **map the data**. This transformation layer assumes that whatever data we're using is valid, which is why the validation has to come first. So now that we have this valid data, we need to make it usable.

Trim whitespace
Lowercase emails
Parse dates
Map fields

So some concrete examples of transformation could be trimming white space in between some content, lowercasing emails or other sorts of inputs, parsing dates to make sure that our application knows what they mean, or mapping specific fields. So since this data has already passed the validation gate, all we need to do is improve upon it to get it into a state in which our specific app can handle it efficiently and optimally.

Keep Validation and Transformation Separate

When you begin to mix both of these validation and transformation, it gets really messy within your application. You want these to be very cleanly separated, no mixing, because that leads to bugs. If you transform before you validate and you flip the sequence(validate then transform → transform then validate), then your backend starts to guess, and we never want our backend guessing. To put it simply, validation just decides if data enters into our system, and transformation obviously transforms and prepares that data to be used.

Validation = Permission
Transformation = Preparation

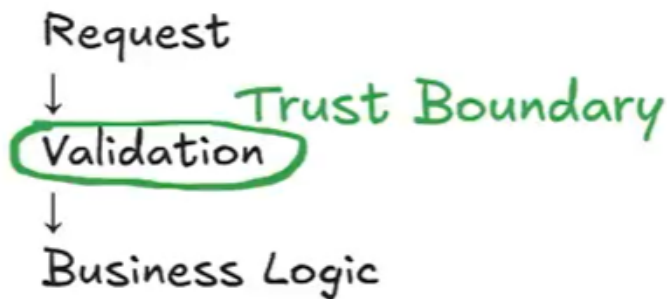
Validation as a Contract

So, now that these are cleanly separated, let's go ahead and focus on validation, and what does it really mean to enforce a contract? Validation isn't necessarily about being super strict, it's just about defining a contract. Essentially, a contract are just restraints or parameters that say, "This is what my system agrees to accept, nothing more, nothing less." And validation is just enforcing the boundary (Required fields, Expected types, Allowed ranges). Data either matches what we have specified in the contract, or it gets rejected. And this rejection is honestly a success. If we reject some invalid data, that means that our validation system is working properly. It simply means that your system is protecting itself.

What Validation Does Not Do

So, I want to clarify a few things that validation does not do. Validation doesn't fix any data, it doesn't guess the intent or mutate any of the values. It simply takes the input as is, and ensures that it matches whatever we're expecting.

Validation at the Trust Boundary



This is a simple example of where validation lives in the pipeline. And honestly, this validation should happen at the edge of the entire life cycle, and should happen as early as possible. And this acts as sort of a trust boundary. Nothing inside your business logic or your app should ever see any sort of untrusted data, ever. If any data that comes in violates a defined validation contract, then it shouldn't enter your system. So, once data passes validation, we trust it.

Transformation Makes Data Usable

Now, we can safely transform it. The validation, right? This just gets it past the gate, but this transformation actually makes the data usable. And now we have valid data. But, it's messy, okay, why? We have this white space in between our delimiters.

The name on the email is in caps, while the actual domain is only capitalized for the first letter, and then .com is fully capitalized. Obviously, this is super messy, and our application might get an error if it tries to use this as is. It's valid data, but it's not in the form that we want it.

Normalizing Data

The transformed version of data will probably look something like: We remove any of this white space. We lowercase everything and make it nice and neat for the backend. And this is called normalizing data. This data has now become normalized. Transformation essentially just standardizes data. It removes any variation, and that means we need it to do no guessing and no validation inside of this transformation phase. Your transformation layer should already assume that the data has been validated. And naturally, that means it shouldn't validate any data, like I said.

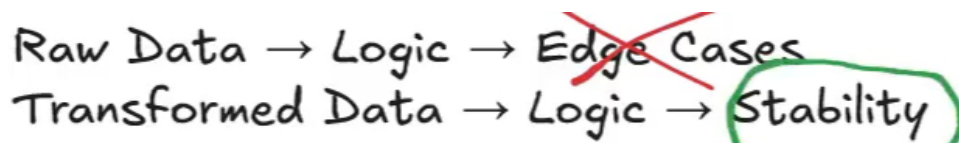
Things like trimming white space, lowercasing strings, parsing numbers or dates, and mapping fields all fall into the transformation phase. This is where the data becomes predictable. And predictability is a good thing in our back end.

External Input vs. Internal Design

Obviously, the external shape of an input should not control our internal design. We accept the data how we accept the data inside of our backend, and we define that. So, we should be able to take any sort of external input and transform that into the format that we expect.

Why Transformation Improves the System

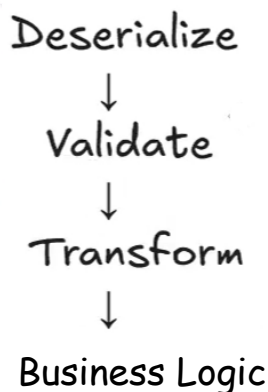
So, how does this actually improve your system? Well, everything downstream gets more simplified when we have clean inputs.



If we just throw some raw data into our logic and run it, then this leads to edge cases and can lead to bugs, errors, the whole line. But if we throw this transformed data, this nicely formed data into our logic, it makes it very stable.

And so the rule I want you guys to remember here is: **transformation reduces variation** of inputs.

The Correct Validation and Transformation Order



So now let's put the two together and lock in the correct order. We have some sort of data that we deserialize into our application, into our memory. We then want to validate the data, make sure that it's good data to be put into our system, and then we want to transform that data into the form in which we expect. And then finally we can throw that into our business logic.

The Clean Pipeline

So here's an example of a bad pipeline. If we just automatically jump into transforming the data, this is bad. And it can look harmless, but obviously you understand why that's not the case. Because just jumping straight into transforming this data can lead to **mask errors** and we're **guessing the intent** of that data and it can **hide invalid inputs**. You're changing data before you even know if it's allowed, which hides the fact if you even know that this data is problematic.

Example: Age Input

Age = " 17 "

Transform → 17

Validate → ?

Age = "seventeen"

Transform → ~~X~~

So here's a concrete example of that. Assume we have this string defined, this string with the name of age, and the contents is 17, but it has this white space between the delimiters. Transformation can make valid data look good, but imagine if this age variable is an actual string of characters and it says 17. If you go to transform that data, you're going to have a problem. Because up here it might be able to transform this into a string, but this("seventeen") is just going to fail. And also with this(17), we have to check the data. We probably have to check, okay, is it a string of numbers? If so, can we change that into an integer? But you shouldn't have to do those checks inside of the transformation phase.

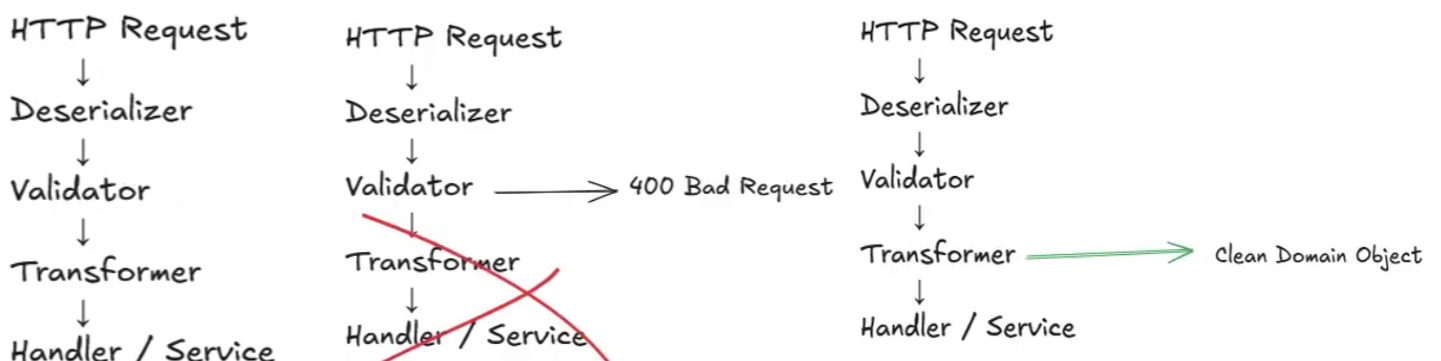
When we validate the data properly, that means that we'll have to do fewer checks down the road.

Validation and Transformation in System Design



So, to tie this into like system design, this is how clean systems are designed and operate. You have an API layer that takes in data. When you get that data, you validate it, transform it, and then and only then you throw it into the domain logic, the business logic. You want to **validate for safety and transform for usability**. You never want to flip the order on these.

Validation and Transformation in Real Code



So, now that we have this order, let's talk about where this shows up in real code. So, before I show any language or actual code, I'm going to start at a high-level overview and then break it down from there. So, every back-end framework does this, even if it does so under the hood. The names might get changed around, but the steps don't. Should this validator throw a bad request and an error, then this data should never reach our handler and never reach our business logic. But, should this get validated and should this be a clean domain object after transformation, that's when we throw it into our handler that then routes it to some business logic.

Separation of Concerns

So, this is the clean separation of concerns. The validation worries about the rules behind what data we're taking in.

The transformation then, assuming that the data follows all of our set rules, should shape the data into something neat and nice and something that the back-end can work with. And then finally, our business logic should actually perform the behavior that we want with this data. Each layer has one job, one concern. **DTOs**, **schemas**, **pipes**, and **middleware**, these are all different ecosystems, but they have the same idea.

Business Logic Assumes Valid Input

And a junior mistake is throwing validation inside of some sort of service, if you're validating deep inside of your code, then you waited a way too long to validate it. **Business logic assumes valid input**. This is the rule I want you guys to understand here. This assumption needs to be earned, not hoped for or inferred. You need to validate it because the business logic will always assume that whatever you're throwing at it is good data.

Common Validation Mistakes

So, now let's talk about where people go wrong and where some small mistakes can cause some real bugs. So, if you've worked on a few back-end code bases before, you've probably seen all or a majority of these.

Mistake 1: Validation as Mutation

So, like I said, **validation isn't a mutation of data**. This logic should not be in your validation layer. Validation isn't there to fix the data. It's only there to decide if this data is usable and if it follows the rules so that we can transform it later.

Mistake 2: Implicit Conversion

The next mistake would be doing some sort of implicit conversion on data without any check. If we silently coerce this data into being a type that we don't even know necessarily whether it should be that type or not, it's going to hide bad input. Probably in most cases, this string 42 should probably be of an integer type, but we don't know that for certain.

Mistake 3: Validating Too Late



Another one and one that I've talked about is validating too late. If you're waiting until after it reaches your service to validate it, then by the time you actually validate the data, the damage has already been done. And this may seem obvious, but you would genuinely be shocked at the amount of times that I've seen this in code bases.

Mistake 4: Revalidating Everywhere

Validate here
Validate here
Validate here

Redundant

Another problem is revalidating everywhere. When you have to validate the same data multiple times, it just becomes redundant. One boundary and one contract for your validation, not multiple sub-validation layers.

Mistake 5: Letting Raw Input Leak



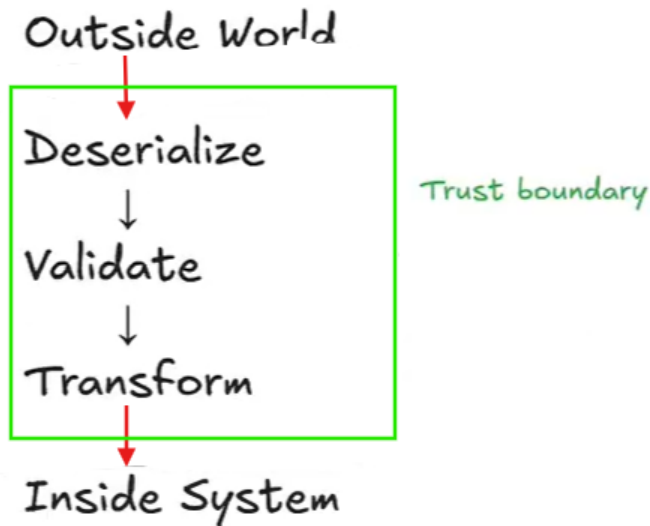
And the last one I want to talk about is letting raw input leak. And this is how bugs become incidents. If you just have some raw data that gets into your database without validation, you're fried, brother. You are cooked. Got to validate it. You never let raw data touch anything.

The Complete Mental Model



So, like I've already shown, you have a request, you need to validate and then transform that data before you can even think about using it. It's simple, it's predictable, and it's just the correct way to do things. Boundaries exist to protect our systems.

The Trust Boundary



So, let's go ahead and wrap this up and merge everything into a single mental model. This right here is our trust boundary. Everything outside of this is untrusted. Everything inside assumes correctness.

Deserialization deals with the structure, validation is for permission, and transformation is for consistency of that data. And inside our system, we give it predictable data. Never skip and never reorder this. If you get the order right, most of your back-end problems will disappear.

Simple Rule of Thumb

Simple rule of thumb, clean inputs create simple systems, and simplicity is a good thing. Complex logic is often a symptom and not a requirement of your system. If you can simplify it down, you're going to have a much better time.

Real-World Example: Zod Validation

```
import { z, ZodError } from 'zod';
import { ValidationError } from './error';

export function validateSchema<T>(schema: z.ZodSchema<T>, data: unknown): T {
  try {
    return schema.parse(data);
  } catch (error) {
    if (error instanceof ZodError) {
      const messages = error.errors.map((e) => `${e.path.join('.')}: ${e.message}`);
      throw new ValidationError(`Validation failed: ${messages.join(', ')}`);
    }
    throw error;
  }
}

export function isValidEmail(email: string): boolean {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return emailRegex.test(email);
}

export function isValidPhoneNumber(phone: string): boolean {
  const phoneRegex = /^[+]?[1-9]\d{7,14}$/;
  return phoneRegex.test(phone);
}

export function sanitizeString(input: string): string {
  return input.trim().replace(/[<>]/g, '');
}
```

I wanted to show you guys a real example from a project of mine. This is the validator file, and ofcourse it's very simple, but there's a lot in these 30-something lines of code. So, I like to use Zod for schema validation, but like I said, it might abstract a lot of the under-the-hood work away, but it's really all the same. So, essentially, we just have to validate our schema and then parse whatever data we're going through to make sure that it fits our schema, right? So, essentially, what this function is do, if you just remove the error catching, is it's just returning the parsing of the data to ensure that whatever schema we have defined matches the data that's coming in.

So, cut me a little bit of slack with this sanitized string function because I know that I said that you should not be transforming any data in your validators, but I was lazy, okay? Do as I say, not as I do, right? But just we have some functions here to check if something is valid, right? Is this email valid? We get the regex of an email and we just test it against that. Same with the phone number. Honestly, should this sanitize string function be separated? Maybe, but this is such a small project that I decided to throw it in there.

Transforming Data in the Retail File

```
export const RetellWebhookEventSchema = z.object({  
  
  event: z.enum(['call.started', 'message.transcribed', 'tool.call', 'call.ended']),  
  
  call_id: z.string(),  
  
  agent_id: z.string(),  
  
  timestamp: z.number(),  
  
});  
  
export const CallStartedEventSchema = RetellWebhookEventSchema.extend({  
  
  event: z.literal("call.started"),  
  
  caller_number: z.string(),  
  
  metadata: z.record(z.unknown()).optional(),  
  
});
```

And then inside of my retail file, I just have various events that's calling Zod to transform this data into the format that we need. So, just converting whatever these inputs are into the proper string type or the proper record, whatever type it is that we need the data to be in.

So, this is supposed to be more of like a beginner course and this is kind of like some intermediate to advanced TypeScript. So, I don't want to get too deep into this, but basically all you need to understand is the concept. But I do want to show some real-world examples in my videos. I don't want it to be just pure theory. So, I just very simply here, right? Like for the retail event hooks, like we define a type and all of the relevant types I've defined here. And then we can go ahead and define these events based on this schema. So, that whenever we call it in the future, this data that we get put in will automatically be transformed into whatever we need it to be.